

Final Result

Review Questions

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	40
41	42	43	44	45	46	47	48	49	50										



Question 1

You are extracting structured findings from code reviews. Despite detailed instructions specifying the output format, results vary across runs—some outputs include all required fields, while others omit or reorder them.

What change would most effectively improve consistency?



Increase prompt length to fully describe the required structure

Incorrect. Longer instructions often reduce clarity. Examples are more effective than verbose descriptions.



Validate outputs after generation and retry if fields are missing

Incorrect. While useful, this is a downstream correction. Few-shot examples improve first-pass consistency.



Add stricter instructions emphasizing that all fields must be present

Incorrect. Stronger wording does not guarantee compliance. The model benefits more from examples than emphasis.



Include 2–3 examples demonstrating the exact expected output format

SELECTED

Correct. Few-shot examples are the most reliable way to enforce consistent formatting. They show the model exactly how outputs should be structured.

Domain 4: Prompt Engineering & Structured Output

✕ Question 2

Your agent retrieves order details using a tool and then determines next steps based on the result. However, in production, the agent repeatedly calls the same tool with identical inputs, failing to progress toward resolution.

Investigation shows that tool results are not included in subsequent requests to the model.

What change would most effectively resolve this issue?

- ✕ Add few-shot examples showing the correct sequence of tool usage and decision-making

Incorrect. Few-shot examples improve patterns but cannot compensate for missing runtime data. The issue is not behavior modeling, but lack of state being passed between iterations.

- ✕ Introduce a rule that prevents the same tool from being called twice with identical inputs

Incorrect. This may block redundant calls but does not solve the underlying reasoning problem. It can also prevent legitimate repeated calls in cases where retries are necessary.

- ✕ Modify the system prompt to remind the model to avoid repeating tool calls unnecessarily

Incorrect. While this may reduce repetition slightly, it does not address the missing context. The model cannot make informed decisions without access to prior tool outputs.

- ✓ Append tool results to the conversation history before the next loop iteration so the model can incorporate new information into its reasoning

Correct. Agentic loops rely on updated context to guide decision-making across iterations. Without including tool results, the model has no awareness of prior actions, leading to repeated calls and stalled progress.

Domain 1: Agentic Architecture & Orchestration

× Question 3

Your agent handles subscription cancellations and refunds. In production, you observe that in some cases the agent calls `issuerefund` before confirming eligibility through `checksubscription_status`, resulting in refunds being issued for inactive or expired accounts.

What is the most effective way to ensure correct workflow ordering?

- × Introduce a delay between tool calls to ensure the agent evaluates eligibility before proceeding

Incorrect. Timing does not control logical execution order. This does not prevent the agent from making incorrect decisions.

- ✓ Add a prerequisite check that blocks `issuerefund` until `checksubscription_status` confirms eligibility

Correct. Programmatic enforcement ensures that critical steps are always completed before downstream actions. This provides deterministic guarantees and eliminates the possibility of skipping required validation.

- × Update the system prompt to clearly state that subscription status must be verified before issuing refunds

Incorrect. Prompt instructions are not guaranteed to be followed in all cases. For critical business rules, relying on probabilistic behavior introduces risk.

- × Add few-shot examples showing the correct sequence of checking status before issuing refunds

Incorrect. Examples can improve behavior but cannot enforce it. The agent may still violate the sequence under certain conditions.

Domain 1: Agentic Architecture & Orchestration

✕ Question 4

Your agent explores a large codebase over many steps. As the session continues, responses become inconsistent and start referencing “typical patterns” instead of actual findings from earlier steps.

You want to preserve accuracy across long exploration sessions.

Which adjustment would best address this?

- ✓ Maintain a scratchpad file with key findings and reference it in future steps

Correct. Externalizing important findings prevents context degradation and allows consistent reasoning across long sessions.

- ✕ Add instructions telling the agent to stay consistent with earlier findings

Incorrect. Instructions cannot compensate for lost context.

- ✕ Increase the number of files analyzed per step to improve coverage

Incorrect. This increases cognitive load and worsens context issues.

- ✕ Restart the session frequently to avoid context overflow

Incorrect. This discards useful knowledge instead of preserving it.

Domain 5: Context Management & Reliability

× Question 5

Your team defines coding standards in a CLAUDE.md file located at `~/claude/CLAUDE.md`. On your machine, Claude consistently follows these conventions. However, when a new team member clones the repository, Claude does not apply the same standards.

What is the most likely cause?

- × The repository does not include enough example files for Claude to learn conventions

Incorrect. Claude relies on explicit configuration, not passive learning from examples.

- × The CLAUDE.md file needs to be referenced using `@import` syntax

Incorrect. Import syntax is for modularization, not for making configurations available across users.

- ✓ The configuration is defined at the user level and is not shared through version control

Correct. Files in `~/claude/CLAUDE.md` apply only to the local user and are not shared with teammates. Project-level configuration must be stored in the repository.

- × The CLAUDE.md file is missing required frontmatter configuration

Incorrect. CLAUDE.md does not require frontmatter to function. The issue is scope, not formatting.

Domain 3: Claude Code Configuration & Workflows

× Question 6

You are building a multi-agent research system with a coordinator and three subagents: search, analysis, and synthesis. During testing, you observe that subagents sometimes call each other directly (e.g., analysis calling search), leading to inconsistent results and difficult-to-debug failures.

What is the most effective fix?



Route all subagent communication through the coordinator agent

Correct. In a hub-and-spoke architecture, the coordinator is responsible for managing all communication, routing, and error handling. This ensures observability, consistent control flow, and prevents unpredictable interactions between subagents.



Limit each subagent to a smaller set of tools to reduce cross-agent calls

Incorrect. While reducing tools may limit misuse, it does not enforce proper communication flow. The core issue is architectural, not tool availability.



Allow subagents to call each other but log all interactions for debugging

Incorrect. Logging improves visibility but does not solve the underlying architectural issue. Direct subagent communication introduces uncontrolled behavior and breaks coordination patterns.



Add prompt instructions discouraging subagents from invoking other subagents

Incorrect. Prompt-based restrictions are not reliable for enforcing architectural constraints. Subagents may still violate instructions under certain conditions.

Domain 1: Agentic Architecture & Orchestration



Question 7

Your agent has access to two tools:

- `get_customer`: "Retrieves customer information"
- `lookup_order`: "Retrieves order details"

During testing, you observe that when users ask about orders, the agent frequently calls `getcustomer` instead of `lookuporder`, even when an order ID is clearly provided.

Which change would most effectively improve tool selection?



Add more examples showing correct tool usage patterns in the system prompt

Incorrect. Examples may help but increase token usage and still rely on inference. Improving descriptions is a more direct and reliable fix.



Rename both tools to shorter names to make them easier to distinguish

Incorrect. Shorter names do not improve clarity of purpose. Without meaningful descriptions, ambiguity remains.



Expand each tool's description to include input formats, example queries, and when each tool should be used

Correct. Tool descriptions are the primary mechanism for tool selection. Clear distinctions, input expectations, and usage boundaries allow the model to reliably choose the correct tool.



Add prompt instructions telling the agent to prioritize order-related tools when an order ID is present

Incorrect. Prompt instructions can influence behavior but do not resolve ambiguity between tools. The root issue is insufficient tool descriptions.

× Question 8

Your CI pipeline includes a step that runs:

```
claude "Review this pull request for performance issues"
```

However, the job never completes. Logs show that Claude Code is waiting for interactive input, causing the pipeline to hang.

What is the correct way to run this command in a CI/CD environment?

× Use a --ci flag: `claude --ci "Review this pull request for performance issues"`

Incorrect. This flag does not exist in Claude Code. The correct approach is to use the documented `-p` flag for non-interactive execution.

✓ Add the `-p` flag: `claude -p "Review this pull request for performance issues"`

Correct. The `-p` (or `--print`) flag runs Claude Code in non-interactive mode. It processes the prompt, outputs the result, and exits immediately without waiting for user input—making it suitable for CI/CD pipelines.

× Set an environment variable `CLAUDENONINTERACTIVE=true` before executing the command

Incorrect. This environment variable does not exist. Claude Code requires explicit CLI flags to control execution mode, not environment-based toggles.

× Pipe an empty input stream into the command: `echo "" | claude "Review this pull request for performance issues"`

Incorrect. While this may prevent immediate blocking in some CLI tools, it does not properly configure Claude Code for non-interactive execution. The process may still behave unpredictably or fail in edge cases.

× Question 9

You are tasked with migrating a large codebase to a new logging library. The change affects multiple modules and requires selecting between several integration approaches. Once the approach is chosen, the implementation steps are straightforward.

What is the most appropriate workflow?

- × Start directly in execution mode since the changes are well-defined

Incorrect. The approach itself is not yet finalized. Skipping planning increases the risk of rework.

- × Switch between plan mode and execution after each step

Incorrect. Frequent switching introduces unnecessary overhead. A phased approach is more effective.

- ✓ Use plan mode first, then switch to direct execution for implementation

Correct. Plan mode is best for evaluating architectural decisions, while execution mode is efficient for applying known changes. Separating these phases improves both accuracy and speed.

- × Perform the entire task in plan mode to maintain consistency

Incorrect. Plan mode is not intended for execution-heavy tasks. It is designed for exploration and decision-making.

× Question 10

Your team wants to create a custom `/analyze` slash command that runs a standardized code quality checklist. This command must be automatically available to all developers when they clone or pull the repository.

Where should you define this command?

- × In a `.claude/settings.json` file with a commands configuration block

Incorrect. This describes a configuration approach that does not exist in Claude Code. Slash commands must be defined as files in the appropriate directory, not through a JSON configuration file.

- ✓ In the `.claude/commands/` directory within the project repository

Correct. This is the proper location for project-scoped slash commands. Files placed here are version-controlled and shared with the entire team. When developers clone or pull the repository, these commands become immediately available, ensuring consistency across the development workflow.

- × In `~/.claude/commands/` on each developer's local machine

Incorrect. This directory is intended for personal, user-scoped commands. Commands stored here are not shared via version control, meaning each developer would need to create and maintain them individually. This does not meet the requirement for team-wide availability.

- × In the root `CLAUDE.md` file alongside project instructions

Incorrect. CLAUDE.md is used to provide guidance, conventions, and contextual instructions to Claude, not to define executable commands. Placing command logic here would not make it available as a slash command.

Domain 3: Claude Code Configuration & Workflows

× Question 11

During evaluation, you notice that your report-generation agent frequently pauses to validate basic details (e.g., dates, names, simple statistics). Currently, it delegates these checks back to the coordinator, which invokes a retrieval agent and then resumes report generation. This adds multiple round trips and increases overall latency by 35%.

Analysis shows that roughly 80% of these validations are simple lookups, while the remaining cases require deeper investigation across multiple sources.

What is the most effective approach to reduce latency while maintaining system reliability?

×

Give the report-generation agent full access to all retrieval tools so it can independently handle all validation scenarios

Incorrect. This over-expands the agent's responsibilities and violates separation of concerns. It increases the likelihood of misuse and reduces the clarity of agent roles within the system.

×

Expand the retrieval agent's initial output to include additional surrounding context in anticipation of potential validation needs

Incorrect. This relies on speculative pre-fetching, which cannot reliably predict what will need verification. It increases context size unnecessarily and does not guarantee that required details will be available when needed.

×

Modify the report-generation agent to collect all validation requests and send them in a single batch to the coordinator after completing its draft

Incorrect. This introduces blocking dependencies, as some validations may be required earlier in the synthesis process. Delaying validation can lead to incorrect intermediate reasoning and reduce output quality.



Provide the report-generation agent with a limited quick_lookup tool for simple validations, while complex checks continue through the coordinator and retrieval agent

Correct. This approach applies the principle of least privilege—giving the agent only the capabilities needed for the common case (simple lookups) while preserving the existing architecture for more complex tasks. It reduces latency significantly without compromising system structure or reliability.

Domain 2: Tool Design & MCP Integration



Question 12

After running your system on the topic “economic impact of renewable energy adoption,” you observe that each subagent completes successfully: the data collection agent retrieves relevant reports, the analysis agent summarizes findings accurately, and the synthesis agent produces a clear final report. However, the output focuses only on solar energy, completely missing wind, hydro, and geothermal sectors.

Reviewing the coordinator’s logs, you find it decomposed the task into: “solar energy job creation,” “solar infrastructure investment,” and “solar market growth.”

What is the most likely root cause?



The coordinator agent’s task decomposition is too narrow, resulting in incomplete coverage of the broader topic

Correct. The coordinator defines the scope of the entire system by determining which subtasks are executed. In this case, it incorrectly narrowed the topic to only solar energy, excluding other major sectors. Since all subagents performed correctly within their assigned scope, the issue originates from how the task was decomposed.



The analysis agent is filtering out non-solar findings due to overly strict relevance criteria

Incorrect. There is no indication that relevant information was filtered out. The absence of non-solar content stems from it never being included in the assigned tasks, not from downstream filtering.



The synthesis agent lacks instructions to detect missing coverage across different energy sectors

Incorrect. While synthesis could potentially identify gaps, it is working only with the data provided. The missing sectors were never explored upstream, so the issue is not with synthesis but with initial task assignment.



The data collection agent is not retrieving diverse enough sources and needs expanded query strategies

Incorrect. The data collection agent is successfully retrieving relevant information based on the queries it was given. The limitation is not in retrieval quality but in the scope of the queries defined by the coordinator.

Domain 1: Agentic Architecture & Orchestration



Question 13

You are using Claude Code to analyze a large codebase. After completing an initial session, several files are modified by your team. You resume the previous session using `--resume`, but the agent produces outdated recommendations based on old file states.

What is the most effective way to handle this situation?



Start a new session and provide a structured summary of prior findings along with updated file context

Correct. When prior tool results are stale, starting fresh with a structured summary ensures the agent works with accurate and relevant information. This avoids relying on outdated context while preserving important insights from earlier analysis.

- ✗ Re-run the same prompts in the resumed session without modifying the context

Incorrect. Without updating the context, the agent will continue producing results based on outdated information. This does not address the root issue of stale data.

- ✗ Continue using the resumed session and rely on the agent to detect changes automatically

Incorrect. The agent does not automatically detect external changes to files unless explicitly informed. This leads to incorrect reasoning based on outdated context.

- ✗ Use `fork_session` to branch from the previous session and continue analysis from there

Incorrect. Forking preserves the same baseline context, including stale data. It is useful for exploring alternatives, not for handling updated inputs.

Domain 1: Agentic Architecture & Orchestration

✗ Question 14

You are designing an agent to perform code reviews. The workflow is well-defined: analyze each file for local issues, then evaluate cross-file interactions for integration problems. The steps are consistent across all pull requests.

Which task decomposition strategy is most appropriate?

- ✗ Provide a high-level goal and let the agent determine all steps without any predefined structure

Incorrect. This removes structure entirely and risks inconsistent or incomplete reviews. Predictable workflows benefit from explicit step sequencing.

✗ Run multiple independent review passes and combine the results at the end

Incorrect. While this may improve coverage in some cases, it does not address the need for structured decomposition. It can also introduce redundancy and inconsistency.

✓ Use a fixed sequential pipeline where each step runs in order, analyzing files individually and then performing a cross-file pass

Correct. Prompt chaining is ideal for predictable, repeatable workflows with clearly defined steps. It ensures consistent execution and avoids unnecessary variability in how tasks are performed.

✗ Allow the agent to dynamically generate tasks based on intermediate findings during the review

Incorrect. Dynamic decomposition is better suited for open-ended or exploratory tasks. For structured workflows like code reviews, it introduces unnecessary variability.

Domain 1: Agentic Architecture & Orchestration

✗ Question 15

Your team is evaluating ways to reduce API costs for automated workflows. Currently, two processes rely on real-time Claude calls:

- (1) a blocking validation step that must complete before code can be deployed, and
- (2) a nightly analytics report that is reviewed the next morning.

A proposal suggests migrating both workflows to a batch processing API to reduce costs.

How should you evaluate this proposal?



Use batch processing for the nightly analytics reports only, and keep real-time calls for the blocking validation step

Correct. Batch processing is well-suited for non-blocking, latency-tolerant workloads like overnight reports. However, blocking workflows—such as deployment validation—require immediate results. Matching each workflow to the appropriate execution model balances cost savings with system reliability.



Migrate both workflows to batch processing with a fallback to real-time execution if results are delayed

Incorrect. This introduces unnecessary complexity and operational overhead. A simpler and more effective approach is to assign each workflow to the appropriate API based on its latency requirements.



Migrate both workflows to batch processing and poll for completion status before proceeding

Incorrect. Polling does not solve the fundamental issue: batch processing has no guaranteed latency. Blocking workflows cannot depend on unpredictable completion times, even if they “usually” finish quickly.



Keep both workflows on real-time calls to ensure consistent execution timing

Incorrect. While this maintains reliability, it ignores the opportunity to reduce costs for workflows that do not require immediate results. The goal is to optimize, not avoid change entirely.

Domain 4: Prompt Engineering & Structured Output

× Question 16

Your system extracts structured data from documents. It reports high overall accuracy, but errors still appear in certain document types. You want to reduce human review while maintaining reliability.

Which approaches best support safe automation? (Select TWO)



Use field-level confidence scores calibrated with validation data

Correct. Calibrated confidence enables selective review and efficient allocation of human effort.



Remove review for all high-confidence outputs immediately

Incorrect. Confidence must be validated and calibrated before being trusted.



Rely on overall accuracy metrics to decide when to remove review

Correct. Calibrated confidence enables selective review and efficient allocation of human effort.



Analyze accuracy by document type and field before reducing review

Correct. Aggregate accuracy can hide weaknesses. Segment-level analysis ensures reliability across all cases.

Domain 5: Context Management & Reliability

× Question 17

You are implementing an agentic loop for a support agent that processes refund requests. In production, you observe that the loop sometimes terminates early before completing required tool calls, even though additional actions are needed.

Your current implementation ends the loop when the assistant response does not contain any tool calls in the message text.

What is the most effective fix?

- × Require the model to include a specific keyword such as "COMPLETE" to signal termination

Incorrect. Parsing natural language for termination signals is an anti-pattern and prone to failure. The model may omit or misuse the keyword, leading to inconsistent behavior.

- × Enforce a minimum number of loop iterations before allowing termination

Incorrect. Arbitrary iteration limits do not reflect actual task completion and can either terminate too early or run unnecessarily long. Loop control should be based on model intent, not fixed counts.

- ✓ Continue the loop based on the `stop_reason`, proceeding when it is "tool_use" and terminating only when it is "end_turn"

Correct. The `stop_reason` is the authoritative signal for controlling agentic loop execution. Using it ensures the loop continues whenever the model intends to call tools and only stops when the model explicitly signals completion, avoiding premature termination.

- × Modify the prompt to instruct the model to explicitly state when all actions are complete before ending the loop

Incorrect. Prompt instructions are not reliable for loop control because they depend on probabilistic model behavior. The system should rely on structured signals like `stop_reason` instead.

Domain 1: Agentic Architecture & Orchestration

× Question 18

A user asks about their order status. The system returns multiple matching orders for the same name. The agent selects the most recent order and proceeds.

Later, the user reports that the response referred to the wrong order.

Which approach would have prevented this issue?

- × Use heuristics to choose the order with the highest value

Incorrect. Heuristics are unreliable and can lead to incorrect conclusions.

- × Return all orders and let the user interpret the results

Incorrect. This shifts responsibility to the user instead of resolving ambiguity.

- ✓ Ask the user for an additional identifier before selecting an order

Correct. When multiple matches exist, clarification is required. Heuristic selection introduces risk of incorrect assumptions.

- × Select the most recent order by default for consistency

Incorrect. Consistency does not equal correctness. The wrong order may still be selected.

Domain 5: Context Management & Reliability

× Question 19

Your agent assigns severity levels (low, medium, high) to detected issues. During evaluation, you observe inconsistent classification—similar issues are sometimes labeled differently across files.

What is the most effective improvement?

- × Reduce the number of severity levels to simplify classification

Incorrect. While this may reduce variability, it does not solve inconsistency in classification logic.

- × Provide more general guidance on how severity should be interpreted

Incorrect. General guidance is insufficient for consistent classification. Specific criteria are required.

- × Add instructions telling Claude to be consistent when assigning severity

Incorrect. Consistency cannot be achieved without clear definitions. The model needs explicit criteria, not general guidance.

- ✓ Define explicit severity criteria with concrete examples for each level

Correct. Concrete definitions eliminate ambiguity and enable consistent classification. Examples help the model generalize severity decisions.

Domain 4: Prompt Engineering & Structured Output

× Question 20

Your agent currently achieves a 58% first-contact resolution rate, significantly below the 80% target. Analysis shows that the agent frequently escalates simple account updates (e.g., address changes with verified identity) while attempting to resolve complex billing disputes that require policy interpretation and exceptions.

What is the most effective way to improve escalation calibration?



Add explicit escalation criteria to the system prompt with few-shot examples illustrating when to escalate versus resolve autonomously

Correct. The issue stems from unclear decision boundaries—specifically, the agent does not reliably distinguish between simple and complex cases. Providing explicit criteria along with concrete examples directly addresses this root cause. This is the most effective and proportionate first step before introducing additional infrastructure.



Use sentiment analysis to detect frustrated users and automatically escalate when negative sentiment is detected

Incorrect. Sentiment reflects customer emotion, not case complexity. A frustrated customer may still have a simple issue that the agent can resolve, while a calm customer may present a complex request requiring escalation. This approach addresses the wrong signal and does not solve the underlying problem.



Require the agent to output a confidence score for each decision and automatically escalate when the score falls below a defined threshold

Incorrect. LLM self-reported confidence is not reliably calibrated. The agent is already making incorrect decisions about case complexity, which means its confidence scores cannot be trusted to guide escalation decisions accurately.



Train a separate classification model on historical support data to determine whether a request should be escalated before processing

Incorrect. While this could potentially improve classification, it is an over-engineered solution at this stage. It requires labeled data, additional infrastructure, and ongoing maintenance.

Prompt-level improvements should be attempted first, as they are simpler and often sufficient.

Domain 5: Context Management & Reliability

× Question 21

Your codebase contains multiple areas with different conventions: frontend components follow one pattern, backend services follow another, and data access layers use a separate structure. Test files are distributed throughout the repository alongside the files they test (e.g., `user.service.test.ts` next to `user.service.ts`), and you want all test files to consistently follow the same conventions regardless of where they are located.

What is the most maintainable way to ensure Claude automatically applies the correct conventions when generating code?

×

Create separate skills in `.claude/skills/` for each code type and include conventions within each skill definition

Incorrect. Skills are designed for on-demand workflows, not for automatically applying conventions during normal code generation. This would require manual invocation or inconsistent usage.

×

Add a `CLAUDE.md` file to each directory with localized rules specific to that part of the codebase

Incorrect. While this can work for directory-specific rules, it does not scale well when conventions need to apply across multiple directories (such as test files spread throughout the repo). It also increases maintenance overhead and fragmentation.

×

Define all conventions in the root `CLAUDE.md` file and rely on Claude to infer which rules apply in each context

Incorrect. This relies on implicit inference rather than explicit rule matching. As the codebase grows, this becomes unreliable and can lead to inconsistent application of conventions.



Create rule files in `.claude/rules/` using YAML frontmatter with glob patterns to apply conventions based on file paths

Correct. Path-specific rules allow conventions to be applied automatically based on file patterns (e.g., `*/.test.ts`). This ensures consistent behavior regardless of where files are located, making it ideal for cases like distributed test files. It is scalable, maintainable, and reduces unnecessary context loading.

Domain 3: Claude Code Configuration & Workflows



Question 22

Your agent summarizes findings from multiple sources. During synthesis, specific claims are included, but their original sources are no longer traceable.

This creates difficulty when verifying accuracy or resolving conflicting data.

Which change would best address this issue?



Combine all sources into a single unified summary format

Incorrect. This removes distinctions between sources and reduces transparency.



Require structured claim-to-source mappings in subagent outputs

Correct. Preserving attribution ensures traceability and supports conflict resolution during synthesis.



Increase the number of sources used during synthesis

Incorrect. More sources increase complexity but do not preserve attribution.



Summarize findings more aggressively to reduce context size

Incorrect. This increases the risk of losing attribution.

Domain 5: Context Management & Reliability



Question 23

Your system uses multiple extraction tools with different schemas depending on document type. You configure the agent with `tool_choice: "auto"`.

During testing, you observe that for some inputs, the agent returns natural language instead of invoking any tool, causing downstream processing failures.

What is the most appropriate configuration change?



Set `tool_choice: "any"` to ensure a tool is always called

Correct. This guarantees that the model invokes a tool instead of returning free-form text, ensuring structured output across varying inputs.



Reduce the number of tools to simplify decision-making

Incorrect. While this may reduce ambiguity, it does not ensure that a tool is used in all cases.



Force a specific tool to guarantee structured output

Incorrect. Forcing a specific tool is only appropriate when the document type is known. In multi-format scenarios, this reduces flexibility.



Add instructions telling the agent to always use tools

Incorrect. Instructions are not reliable for enforcing tool invocation. Configuration must guarantee behavior.

Domain 4: Prompt Engineering & Structured Output

✕ Question 24

Your agent handles long-running customer conversations involving multiple orders. Over time, details like order IDs and refund amounts become inconsistent in later responses.

You observe that earlier facts are summarized into shorter descriptions as the conversation progresses.

Which change would best preserve accuracy?

- ✓ Maintain a structured “case facts” block with key details passed in every prompt

Correct. Persisting critical facts outside summaries ensures they are not degraded or lost over time. This avoids issues caused by progressive summarization.

- ✕ Summarize the conversation more frequently to reduce context size

Incorrect. More summarization increases the risk of losing important details.

- ✕ Increase the maximum context window to retain more conversation history

Incorrect. Larger context helps, but does not prevent information degradation through summarization.

- ✕ Add instructions reminding the agent to remember important details

Incorrect. Instructions alone do not guarantee retention of precise facts like IDs or amounts.

× Question 25

A pull request introduces changes across 16 files in a payment processing module. Your current automated review analyzes all files in a single pass, but results are inconsistent: some files receive detailed feedback while others are barely reviewed, critical issues are missed, and similar patterns are flagged differently across files.

How should you restructure the review process?

×

Use a larger-context model to improve coverage and consistency when analyzing all files together in one pass

Incorrect. Increasing context size does not guarantee better attention or consistency. The issue is not capacity, but how attention is distributed across many files.

×

Run multiple review passes across the full change set and report only issues that appear consistently across runs

Incorrect. This may reduce noise but risks suppressing real issues that are detected inconsistently. It trades recall for perceived precision, which is not suitable for reliable code review.

×

Require developers to split large pull requests into smaller groups of changes before running automated review

Incorrect. While smaller PRs can help in practice, this shifts responsibility to developers and does not fix the underlying limitation in how the review system operates.

✓

Analyze each file individually for local issues, then run a separate pass to evaluate cross-file interactions and data flow

Correct. This approach addresses attention dilution by separating concerns. Per-file analysis ensures consistent depth, while a dedicated integration pass captures cross-file dependencies and inconsistencies.

Domain 4: Prompt Engineering & Structured Output

✕ Question 26

Your agent must choose between two tools:

- one for retrieving structured data
- one for summarizing content

For clear inputs, tool selection works well. However, for ambiguous requests like “get insights from this report,” the agent inconsistently chooses between tools.

What is the most effective way to improve reliability?

- ✕ Reduce the number of available tools to simplify selection

Incorrect. This may reduce complexity but does not address how to handle ambiguous cases.

- ✓ Provide a few targeted examples showing how ambiguous requests should be handled

Correct. Few-shot examples demonstrate decision-making in ambiguous cases, allowing the model to generalize patterns beyond explicit instructions.

- ✕ Add instructions describing how to choose between the tools

Incorrect. Instructions often fail in ambiguous situations. Examples provide stronger guidance through demonstration.

- ✕ Rename the tools to better reflect their purpose

Incorrect. Naming helps clarity but does not resolve ambiguity in decision-making.

Domain 4: Prompt Engineering & Structured Output

✕ Question 27

account closure. Your agent attempts to handle everything in a single pass but produces incomplete and unclear responses. In some cases, escalation to a human agent lacks sufficient context, forcing manual re-investigation.

Which changes would most effectively improve both resolution quality and escalation handoffs? (Select TWO)

- ☒ Break the request into distinct concerns, investigate each independently, and then combine results into a unified response

Correct. Decomposing multi-concern requests ensures each issue is handled thoroughly without interference. This improves completeness and allows parallel investigation when appropriate.

- ☒ Include a structured handoff summary with key details such as customer ID, issue type, root cause, and recommended actions

Correct. Human agents often lack access to full context, so structured summaries are essential for efficient escalation. This reduces rework and ensures continuity of resolution.

- ☐ Instruct the agent to prioritize resolving the most urgent issue first and ignore secondary concerns

Incorrect. Ignoring additional concerns leads to incomplete resolutions and poor user experience. All relevant issues should be addressed or clearly handled.

- ☐ Allow the human agent to access the full conversation transcript instead of providing a summary

Incorrect. Relying on transcripts increases cognitive load and slows down resolution. Structured summaries are more efficient and reliable for handoff.

Domain 1: Agentic Architecture & Orchestration

✕ Question 28

Your MCP tool occasionally fails when retrieving order details due to invalid input or service issues. Currently, it returns a generic response: "Operation failed."

During testing, the agent repeatedly retries failed requests or provides unclear responses to users.

What is the most effective improvement?

- ✕ Log errors internally but continue returning the same generic message to the agent

Incorrect. Internal logging does not help the agent recover or respond appropriately. The agent needs structured information in the response itself.

- ✓ Return structured error metadata including error category, retryability, and a clear description

Correct. Structured errors allow the agent to distinguish between failure types and decide whether to retry, escalate, or communicate with the user. This enables proper recovery behavior and avoids unnecessary retries.

- ✕ Add prompt instructions telling the agent how to respond when a tool fails

Incorrect. Without structured error data, the agent cannot reliably determine what kind of failure occurred. Instructions alone are insufficient.

- ✕ Add examples showing how to handle failed tool responses

Incorrect. Examples improve behavior but do not provide the necessary information about the error. The agent still lacks context for decision-making.

Domain 2: Tool Design & MCP Integration

✕ Question 29

Your tools are well-defined with clear descriptions and boundaries. However, you notice that the agent consistently selects a specific tool whenever certain keywords appear in user queries, even when another tool would be more appropriate.

Investigation reveals that the system prompt contains keyword-based instructions such as:

"If the user mentions 'summary', use `summarize_content`."

What is the most effective way to fix this issue?

- ✓ Remove or revise keyword-sensitive instructions in the system prompt that override tool descriptions

Correct. Keyword-based prompt instructions can bias tool selection and override well-designed descriptions. Removing these ensures the model relies on tool definitions rather than brittle keyword matching.

- ✕ Increase the number of tools available so the agent has more options to choose from

Incorrect. Adding more tools increases decision complexity and may worsen selection reliability.

- ✕ Add more examples reinforcing correct tool selection in the system prompt

Incorrect. This increases prompt complexity without addressing the root cause. The issue is conflicting instructions, not lack of examples.



Rename tools to better match the keywords used in user queries

Incorrect. This reinforces the flawed keyword-based behavior instead of fixing it. Tool selection should be based on meaning, not keyword matching.

Domain 2: Tool Design & MCP Integration



Question 30

Your coordinator agent currently routes every request through all subagents in sequence: search → analysis → synthesis. However, you notice inefficiencies when handling simple queries (e.g., fact lookups), where invoking all agents adds unnecessary latency and cost.

What is the most effective improvement?



Introduce caching at the synthesis stage to reduce repeated processing

Incorrect. Caching may improve performance in some cases, but it does not address the unnecessary invocation of subagents for simple queries.



Update the coordinator to evaluate the query and invoke only the subagents required for that task

Correct. The coordinator should dynamically decide which subagents to invoke based on task complexity. This reduces unnecessary work, lowers latency, and improves efficiency without compromising system design.



Keep the fixed pipeline but optimize each subagent to respond faster

Incorrect. While performance improvements help, they do not address the inefficiency of invoking unnecessary agents. The core issue is inappropriate task routing, not speed.



Add prompt instructions encouraging subagents to return early when tasks are simple

Incorrect. Subagents should not be responsible for deciding whether they should have been invoked. This logic belongs to the coordinator, not individual agents.

Domain 1: Agentic Architecture & Orchestration



Question 31

Your agent uses an MCP tool to fetch customer data. During testing, you observe that the agent retries requests even when the input is invalid (e.g., malformed customer ID), resulting in repeated failures and wasted computation.

The tool currently does not distinguish between different types of errors.

Which change would most effectively improve the agent's behavior?



Add prompt instructions telling the agent to retry only when necessary

Incorrect. Without structured error signals, the agent cannot reliably determine when a retry is appropriate. Instructions alone are insufficient.



Allow the agent to decide retry behavior based on its internal reasoning

Incorrect. The agent does not have enough information to make correct retry decisions without structured error metadata. This leads to inconsistent behavior.



Include a retryable flag in error responses to indicate whether the failure should be retried

Correct. Explicitly marking errors as retryable or non-retryable allows the agent to avoid unnecessary retries and respond appropriately. This prevents wasted computation and improves efficiency.



Increase the retry limit so the agent has more chances to recover from errors

Incorrect. Increasing retries does not fix incorrect retry behavior. It may worsen inefficiency when errors are not recoverable.

Domain 2: Tool Design & MCP Integration



Question 32

Your system includes separate subagents:

- a search agent with web retrieval tools
- a synthesis agent responsible for combining results

During testing, you observe that the synthesis agent occasionally attempts to call web search tools instead of relying on the search agent's output.

What is the most effective way to prevent this issue?



Restrict the synthesis agent's tool access to only those relevant to its role

Correct. Scoped tool access enforces specialization and prevents misuse. Each agent should only have access to the tools required for its responsibilities.



Allow both agents to access all tools for flexibility

Incorrect. This increases misuse and breaks the separation of responsibilities between agents.



Add prompt instructions telling the synthesis agent not to perform searches

Incorrect. Prompt instructions are not reliable for enforcing behavior. The agent may still attempt to use restricted tools.



Add examples showing that only the search agent should call web tools

Incorrect. Examples guide behavior but do not enforce constraints. The issue is access, not understanding.

Domain 2: Tool Design & MCP Integration



Question 33

Your support agent can issue account credits through an MCP tool. Company policy requires that any credit above \$300 must be reviewed by a human agent before it is approved. During testing, you find that the agent usually follows this rule when instructed in the system prompt, but occasionally still calls the credit tool directly for amounts above the threshold.

What is the most effective way to guarantee compliance with this policy?



Add few-shot examples showing the agent escalating high-value credit requests instead of approving them

Incorrect. Prompt instructions can improve behavior, but they do not guarantee compliance. For policy-critical actions, probabilistic guidance is not sufficient.



Require the agent to explain its reasoning before every credit decision so policy violations can be caught more easily

Incorrect. Explanations may improve observability, but they do not prevent incorrect actions from being taken. The requirement is guaranteed enforcement, not easier post-hoc review.



Add a tool interception hook that blocks credit requests above \$300 and redirects those cases to a human review workflow

Correct. A hook provides deterministic enforcement by intercepting the outgoing tool call before the action occurs. This guarantees policy compliance and allows the workflow to redirect high-value cases into the appropriate human review path.

×

Strengthen the system prompt to emphasize that credits above \$300 always require human approval

Incorrect. Prompt instructions can improve behavior, but they do not guarantee compliance. For policy-critical actions, probabilistic guidance is not sufficient.

Domain 1: Agentic Architecture & Orchestration

×

Question 34

Your synthesis agent has access to 18 tools, including search, parsing, summarization, and data extraction tools. During testing, you observe inconsistent behavior: the agent frequently selects incorrect tools or uses tools outside its intended purpose.

What is the most effective improvement?

×

Provide examples showing correct tool usage patterns

Incorrect. Examples improve guidance but increase token usage and do not reduce decision complexity. The core issue remains.

×

Increase the number of tools so the agent has more options to choose from

Incorrect. Adding more tools worsens the problem by increasing ambiguity and selection difficulty.

✓

Limit the synthesis agent's tool set to only those required for its role

Correct. Reducing the number of available tools lowers decision complexity and improves selection reliability. Agents perform best when given a focused set of tools aligned with their responsibilities.

- ✗ Add prompt instructions clarifying which tools should be used in different situations

Incorrect. Instructions help but do not solve the underlying issue of too many competing options. The agent may still misselect tools.

Domain 2: Tool Design & MCP Integration

✗ Question 35

You are building an agent to add comprehensive test coverage to a large legacy codebase. The structure is unclear, dependencies are not well documented, and the agent must determine where to focus first.

Which approaches would most effectively improve task decomposition? (Select TWO)

- ✓ Allow the agent to dynamically adapt its plan as new dependencies and patterns are discovered during execution

Correct. Dynamic decomposition enables the agent to refine its approach as new information becomes available. This is essential for complex, evolving tasks.

- ✗ Generate tests for all files in parallel without analyzing dependencies or relationships first

Incorrect. This ignores code relationships and risks creating incomplete or low-value tests. Effective decomposition requires understanding dependencies before execution.

- ✗ Define a fixed sequence of steps upfront and require the agent to follow them regardless of findings

Incorrect. Fixed pipelines are not suitable for open-ended problems where the correct approach depends on discovered information. This can lead to inefficient or incorrect outcomes.



Start by mapping the codebase structure and identifying high-impact areas before generating a prioritized plan

Correct. For open-ended tasks, understanding structure and impact is critical before taking action. This ensures effort is focused on the most important areas first.

Domain 1: Agentic Architecture & Orchestration



Question 36

Your project has a large CLAUDE.md file containing testing standards, API conventions, deployment rules, and coding guidelines. Over time, the file becomes difficult to maintain, and developers report inconsistent behavior when working in different parts of the codebase.

What is the most effective way to improve this setup?



Duplicate relevant sections of CLAUDE.md into each subdirectory

Incorrect. Duplication creates maintenance overhead and risks inconsistencies across files.



Move all configuration to a user-level CLAUDE.md file

Incorrect. This removes shared configuration and makes team-wide consistency impossible.



Add more comments to the existing CLAUDE.md file to improve readability

Incorrect. Comments may help readability but do not address structural complexity or scaling issues.



Split the configuration into topic-specific files in `.claude/rules/` and organize them by concern

Correct. Modularizing configuration improves maintainability and allows rules to be applied more selectively. This reduces cognitive load and improves consistency.

Domain 3: Claude Code Configuration & Workflows



Question 37

Production logs show that your agent frequently calls `fetchprofile` when users request shipment updates (e.g., “where is my package #78910”), instead of calling `trackshipment`. Both tools currently have minimal descriptions (“Retrieves user profile data” / “Retrieves shipment status”) and accept similar identifier inputs.

What is the most effective first step to improve tool selection reliability?



Implement a preprocessing classifier that inspects user queries and routes them to the correct tool based on keywords and patterns

Incorrect. This is an over-engineered solution that bypasses the model’s built-in reasoning capabilities. It introduces additional complexity and brittleness, especially for ambiguous or edge-case queries, when the problem can be solved more simply at the tool design level.



Merge both tools into a single `lookup_record` tool that determines internally whether to return profile or shipment data

Incorrect. While consolidation can reduce ambiguity in some designs, it requires significant architectural changes and added internal logic. This is not appropriate as a first step when the immediate issue is poor tool descriptions.



Add few-shot examples to the system prompt demonstrating correct routing of shipment-related queries to track_shipment

Incorrect. Few-shot examples can improve consistency but do not resolve ambiguity in tool definitions. The issue is not a lack of examples—it is that the tools themselves are not clearly differentiated. This adds overhead without fixing the underlying problem.



Expand each tool's description to include supported input formats, example queries, edge cases, and clear boundaries for when each tool should be used

Correct. Tool descriptions are the primary signal the model uses to decide which tool to call. When tools have overlapping inputs and vague descriptions, the model lacks the context needed to reliably differentiate between them. Improving descriptions directly addresses the root cause with a simple, high-leverage fix.

Domain 2: Tool Design & MCP Integration



Question 38

A system processes thousands of documents each day to generate internal insights. The results are reviewed the next morning, and there are no real-time requirements.

The current implementation uses real-time API calls, leading to increasing operational costs.

Which adjustment aligns best with the system's requirements?



Introduce caching to reduce repeated requests

Incorrect. Caching helps with repeated inputs, but this workload involves new documents. It does not address the primary cost driver.



Limit the number of documents processed per run

Incorrect. This reduces throughput rather than improving efficiency. It does not solve the underlying issue.



Switch to batch processing for this workload

Correct. Batch processing is well-suited for high-volume, latency-tolerant scenarios. It reduces cost without impacting usability.



Keep real-time processing to maintain predictable execution

Incorrect. Real-time guarantees are unnecessary here and increase cost without providing value.

Domain 4: Prompt Engineering & Structured Output



Question 39

You've been assigned to migrate a legacy monolithic application into a modular service-based architecture. This effort will impact dozens of files and requires careful decisions around service boundaries, shared dependencies, and data flow between components.

Which approach should you take?



Use direct execution with detailed upfront instructions specifying the exact structure and responsibilities of each module

Incorrect. This assumes you already have a complete and correct understanding of the system's structure without first exploring it. For complex systems, this often leads to flawed designs because hidden dependencies and constraints are not considered.



Start in direct execution mode and switch to plan mode only if unexpected complexity arises during the process

Incorrect. The complexity is already known upfront—this is a large-scale architectural change. Delaying the use of plan mode increases the likelihood of making premature decisions and introducing costly mistakes before fully understanding the system.



Enter plan mode to explore the codebase, understand dependencies, and design an implementation approach before making changes

Correct. Plan mode is specifically designed for complex, multi-file changes that involve architectural decisions and multiple valid approaches. It allows you to safely explore the codebase, map dependencies, and design a structured approach before making irreversible changes, reducing the risk of rework.



Begin with direct execution and refactor components incrementally, allowing structure to emerge during implementation

Incorrect. While incremental changes can work for small tasks, this approach is risky for large architectural transformations. Discovering dependencies late can lead to inconsistent design decisions and significant rework.

Domain 3: Claude Code Configuration & Workflows



Question 40

You instruct Claude to “review code for issues and report any problems.” During testing, the agent flags many minor style issues and local conventions, overwhelming developers and reducing trust in the tool.

What is the most effective way to improve precision?



Provide examples of code with no issues to reduce over-reporting

Incorrect. While helpful, examples alone do not define clear boundaries. The agent may still report irrelevant issues.



Increase the strictness of the prompt to reduce unnecessary findings

Incorrect. Vague tightening of instructions does not clarify expectations. Specific criteria are required.



Define explicit criteria specifying which issue categories to report and which to ignore

Correct. Explicit criteria directly control what is reported, reducing false positives. This improves precision and restores developer trust.



Add instructions telling Claude to only report high-confidence issues

Incorrect. Confidence-based instructions are vague and inconsistently applied. They do not define what should or should not be reported.

Domain 4: Prompt Engineering & Structured Output



Question 41

You are using Claude to transform API responses into a standardized JSON format. Your current prompt describes the transformation rules in detail using natural language.

During testing, outputs are inconsistent—some responses follow the expected structure, while others vary in field names and formatting.

What is the most effective way to improve consistency?



Add instructions emphasizing strict adherence to the format

Incorrect. Emphasis does not resolve ambiguity. Without concrete examples, the model may still produce inconsistent outputs.

- ✓ Provide 2–3 concrete input/output examples demonstrating the expected transformation

Correct. Examples are the most effective way to communicate precise transformation expectations. They reduce ambiguity and allow the model to reliably reproduce the desired structure.

- ✗ Add more detailed explanations describing each transformation rule

Incorrect. More prose increases complexity without improving clarity. The model may still interpret rules inconsistently.

- ✗ Increase the number of iterations so Claude can refine its output

Incorrect. Iteration does not fix unclear instructions. The model needs clearer guidance, not repetition.

Domain 3: Claude Code Configuration & Workflows

✗ Question 42

Your CI pipeline uses Claude Code to analyze pull requests and automatically post inline comments. Currently, Claude outputs free-form text, making it difficult to reliably map findings to specific files and lines.

What change would best improve this?

- ✗ Run multiple passes and compare results

Incorrect. Repetition does not guarantee consistency. The issue is lack of enforced structure, not variability across runs.

- ✓ Use `--output-format json` with a defined schema

Correct. Structured output ensures consistent, machine-readable results. A schema enforces predictable formatting for reliable CI integration.

✗ Add prompt instructions for consistent formatting

Incorrect. Prompt formatting is not guaranteed and can vary across runs. It is not reliable for automation.

✗ Parse the output using regex patterns

Incorrect. Regex-based parsing is fragile and can break with slight formatting changes. It is not suitable for structured automation.

Domain 3: Claude Code Configuration & Workflows

✗ Question 43

You are building a coordinator agent that delegates to search, analysis, and synthesis subagents. During testing, you observe that the synthesis agent produces incomplete reports, missing key findings that were already retrieved by earlier agents.

Investigation shows that the coordinator invokes subagents correctly, but passes only minimal instructions without including prior results.

Which changes would most effectively resolve this issue? (Select TWO)

✗ Add instructions telling subagents to request missing information from other agents when needed

Incorrect. Subagents do not communicate directly with each other in a properly designed system. All coordination should be handled through the coordinator, not delegated to subagents.

✗ Rely on the model to recall prior steps automatically across subagent invocations

Incorrect. Subagents operate with isolated context and do not retain memory between invocations. Assuming implicit memory leads to incomplete or incorrect outputs.



Include complete outputs from prior subagents directly in the prompt when invoking the next subagent

Correct. Subagents do not share memory or inherit context automatically, so all relevant findings must be explicitly passed. Without this, downstream agents lack the information needed to produce complete results.



Structure context passed between agents to separate content from metadata such as sources and timestamps

Correct. Using structured formats preserves important details like attribution and improves clarity for downstream reasoning. This ensures that synthesis agents can accurately interpret and use prior results.

Domain 1: Agentic Architecture & Orchestration

✕ Question 44

Your coordinator agent is responsible for delegating research tasks to multiple subagents. Currently, it invokes each subagent sequentially across separate turns, resulting in increased latency for broad research queries.

You want to improve performance by enabling parallel execution of subagents.

What is the most effective way to achieve this?



Emit multiple Task tool calls within a single coordinator response to spawn subagents in parallel

Correct. The Task tool enables spawning subagents, and issuing multiple calls in a single response allows them to run concurrently. This reduces latency and improves performance for workflows that can be parallelized.

✗ Configure each subagent to automatically trigger other subagents when additional work is needed

Incorrect. This breaks the coordinator-based architecture and introduces uncontrolled behavior. Subagent orchestration should remain centralized for consistency and observability.

✗ Invoke each subagent in separate turns but reduce the prompt size to improve execution speed

Incorrect. While smaller prompts may slightly improve speed, the core issue is sequential execution. This approach does not enable true parallelism.

✗ Add prompt instructions encouraging subagents to work faster when handling requests

Incorrect. Execution speed is not controlled through prompting, and this does not change the sequential nature of the workflow. It does not address the architectural limitation.

Domain 1: Agentic Architecture & Orchestration

✗ Question 45

You are building an agent to extract structured invoice data. The output must always be valid JSON so it can be consumed by downstream systems.

Currently, the agent returns mostly correct JSON, but occasionally produces malformed output or mixes text with JSON.

What is the most effective solution?

✗ Provide examples of correctly formatted JSON outputs

Incorrect. Examples improve consistency but do not guarantee valid syntax. Errors may still occur.

✗ Post-process the output to fix formatting errors

Incorrect. This adds complexity and does not prevent errors at the source. Tool use provides a cleaner solution.

✓ Use tool_use with a defined JSON schema to enforce structured output

Correct. Tool use guarantees schema-compliant output and eliminates JSON syntax errors. It is the most reliable way to ensure structured responses.

✗ Add instructions telling the model to always return valid JSON

Incorrect. Instructions alone are not reliable. The model may still produce invalid or mixed outputs.

Domain 4: Prompt Engineering & Structured Output

✗ Question 46

Your agent extracts structured data using a defined schema. Most responses are valid, but some fail validation due to missing fields or incorrect formats.

Right now, when validation fails, the system simply retries the same prompt. This sometimes succeeds, but failures still occur inconsistently.

Which change best addresses the root cause of these failures?

✗ Switch to a different model configuration for retries

Incorrect. Model changes do not address the absence of corrective feedback. The issue lies in the retry strategy.

- ✓ Include validation error details in the retry prompt so the model can correct specific issues

Correct. Supplying concrete validation feedback enables targeted corrections. This turns retries into guided fixes rather than repeated guesses.

- ✗ Increase the number of retries to improve the likelihood of success

Incorrect. Repeating the same prompt does not resolve the underlying issue. It introduces variability without improving correctness.

- ✗ Strengthen the instructions to emphasize strict schema compliance

Incorrect. Stronger wording does not provide actionable feedback. The model still lacks clarity on what failed.

Domain 4: Prompt Engineering & Structured Output

✗ Question 47

A production audit reveals that in approximately 9% of cases, your agent directly calls `issue_credit` using a customer's email address without first confirming account ownership through `verify_identity`. This has resulted in credits being applied to the wrong accounts when duplicate or outdated emails are provided.

What change would most effectively address this reliability issue?

- ✗ Add few-shot examples demonstrating correct sequencing where `verify_identity` is always called before any account modification

Incorrect. Few-shot examples can improve consistency, but they do not guarantee compliance. In high-risk operations like issuing credits, relying on learned patterns instead of enforced constraints still allows for edge-case failures, especially under ambiguity or conflicting inputs.



Implement a pre-processing step that filters incoming requests and routes only high-confidence identity matches to `issue_credit`

Incorrect. This approach attempts to reduce risk by filtering inputs but does not enforce the required verification step. It also introduces additional complexity and potential blind spots (e.g., incorrectly classified requests), while still allowing unsafe tool execution paths. The core issue is missing enforcement of tool ordering, not input routing.



Add a programmatic prerequisite that blocks `issue_credit` and `fetch_account` calls until `verify_identity` returns a confirmed account ID

Correct. This directly enforces the required tool sequence at the system level, ensuring that identity verification always occurs before any account modification. Because this is a critical business rule with financial impact, deterministic enforcement is required. This eliminates the failure mode entirely rather than attempting to influence model behavior.



Enhance the system prompt to emphasize that identity verification must always occur before issuing credits

Incorrect. While clearer instructions may reduce the frequency of errors, prompt-based guidance remains probabilistic. The agent is already violating this rule in production, which demonstrates that prompting alone is insufficient for enforcing critical workflows where correctness must be guaranteed.

Domain 1: Agentic Architecture & Orchestration



Question 48

You are using Claude to generate validation logic for a form processing system. The initial implementation works for basic cases but fails on edge cases such as null values and invalid formats.

You want to systematically improve the implementation while ensuring correctness across all scenarios.

Which approach would be most effective?

✗ Provide a high-level description of all possible edge cases in the prompt

Incorrect. High-level descriptions lack precision and do not ensure correct implementation. Concrete tests are more effective.

✓ Write test cases covering expected behavior and edge cases, then iterate by providing failing test results

Correct. Test-driven iteration provides concrete feedback that guides the model toward correct behavior. It enables systematic improvement and ensures edge cases are handled properly.

✗ Add more instructions describing how edge cases should be handled

Incorrect. Natural language descriptions can still be interpreted inconsistently. Tests provide precise and verifiable expectations.

✗ Regenerate the implementation multiple times until it produces the correct behavior

Incorrect. Repetition without feedback does not guarantee improvement. The model needs specific signals about what is wrong.

Domain 3: Claude Code Configuration & Workflows

✗ Question 49

During execution of your research pipeline on a complex topic, the `document_fetch` subagent fails due to a rate limit error when querying an external API. The subagent had already retrieved partial results before failing. The coordinator must decide whether to retry, switch strategies, or continue with incomplete data.

Which approach best enables the coordinator to make an effective recovery decision?



Propagate the raw exception to the coordinator and terminate the workflow immediately to avoid inconsistent results

Incorrect. Terminating the workflow prematurely ignores opportunities for recovery. Many failures are partial or recoverable, and shutting down the entire process reduces system resilience without addressing the underlying issue.



Return structured error information to the coordinator including error type, attempted request details, partial results, and suggested fallback strategies

Correct. Structured error context allows the coordinator to make informed decisions about recovery—such as retrying with a modified request, using alternative sources, or proceeding with partial data. This approach preserves transparency and enables intelligent orchestration.



Implement automatic retries within the subagent and return a generic “request failed” message only if all retries are exhausted

Incorrect. While retries can help with transient failures, returning a generic error removes important context. The coordinator loses visibility into what was attempted and what partial results exist, limiting its ability to choose the best next step.



Catch the error and return the partial results as a successful response without indicating any failure

Incorrect. This suppresses the error entirely and misrepresents the state of the system. The coordinator cannot distinguish between complete and incomplete results, which can lead to inaccurate or incomplete final outputs.

Domain 5: Context Management & Reliability



Question 50

Your agent has access to two tools:

- analyze_content: “Analyzes content for insights”
- analyze_document: “Processes documents for information”

During testing, the agent inconsistently switches between these tools for similar inputs, sometimes producing redundant or conflicting outputs.

What is the most effective way to resolve this issue?



Add more examples demonstrating correct tool selection

Incorrect. Examples help but do not eliminate overlap. The underlying ambiguity in tool purpose remains unresolved.



Keep both tools but rely on the synthesis step to reconcile differences

Incorrect. This allows inefficiency and inconsistency upstream. The problem should be fixed at the tool design level.



Add prompt instructions specifying which tool should be used for each type of input

Incorrect. Prompt instructions are less reliable than well-defined tool boundaries. They can be ignored or misinterpreted by the model.



Redesign the tools with clearly differentiated purposes and update descriptions to reflect distinct responsibilities

Correct. Overlapping tool functionality creates ambiguity and inconsistent selection. Clear separation of responsibilities ensures predictable and reliable tool usage.